

Layaway on Shopify — Solution Brief

WORKING PROTOTYPE

Status: live on `se-shopvip-en-cten` — POS · online store · customer accounts · admin · **Architecture:** "Option C — Order Edit as the Installment Engine" · native · single store · no new platform primitives

01 What it is

A layaway (reserve) program that works across every Shopify surface with **one order per layaway**: a customer puts down a deposit (minimum 10%) on a full-price cart — online or at the register — then pays the balance in any number of installments, of **any amount, through any channel**: their account page, an emailed pay link, or in store at POS. The item ships only when the balance reaches zero.

02 How it works — the 60-second version

1. **Deposit.** The customer commits to a layaway in a "down payment" flow (a storefront modal online; a smart-grid tile modal on POS). Both enforce the 10% minimum, show the math (due today vs. remaining), and stamp the cart with identical `layaway_*` attributes. Checkout then charges **only the deposit** — online via a Cart Transform function that reprices the lines; on POS via a fixed discount equal to the balance, so the cashier simply rings a normal sale whose total *is* the deposit.
2. **One order, one ledger.** The resulting order is "deposit-shaped": its total reflects money actually charged so far. A backend app tags it `layaway`, records the **real total, amount paid, and balance in a ledger metafield**, and places a fulfillment hold — *ships at full payoff*.
3. **Installments.** Each payment starts with the customer (or cashier) choosing an amount. The app's installment engine uses the **Order Edit API** to add a custom line item for exactly that amount — which becomes the order's outstanding balance. Shopify's native payment surfaces then do what they always do: **Pay now** online charges exactly that amount; **Collect payment** on POS takes it by tap-to-pay, card, or cash. Repeat until the ledger hits zero; the hold releases automatically.
4. **Visibility.** A customer sees their installment plan (balance, history, "Make a payment") on their account's order page. Staff see every layaway — channel, status, paid vs. balance — in a dashboard inside the app in admin, and can look up any layaway on POS by customer email or order number.

03 Why this design

Shopify has **no native partial-payment option for online customers**: the only thing a customer can ever pay against an order is its *entire* outstanding amount. That single constraint drives the whole architecture.

Orders therefore start small (the deposit) and **grow with each payment** — every installment creates a bite-sized outstanding that native checkout can charge exactly. The order total becomes a record of payments; the **ledger metafield is the source of truth** for the real total and remaining balance.

04 Caveat: where we started on POS, and where we landed

The original POS approach was more native — and incompatible. Our first build used POS's built-in partial payment: ring the full \$1,000 cart, capture \$250 at the register, and the order lands as genuinely "partially paid" with \$750 outstanding. It was the cleanest possible in-store experience with perfect order data.

It failed the moment the customer went home. That order's \$750 outstanding is indivisible online — Pay now demands all of it. In-store-started layaways couldn't be paid down in installments from the customer's account, which broke the core promise of the program.

The fix was unifying the order shape. POS layaways are now created exactly like web ones: the extension applies a fixed discount equal to the balance, the deposit is charged as a normal full payment, and the order is indistinguishable from a web reserve. A bonus we didn't expect: the cashier experience got *simpler* — no partial-tender steps, no amounts to remember; the total on the screen is the deposit.

Trade-offs we accepted (flagged in the design from day one):

- Order totals no longer equal merchandise value mid-layaway → **reporting must read the ledger**, not order totals.
- A "Layaway — balance due later" discount line is visible on POS-created orders and every subsequent pay screen (it is the one-time deposit discount — never a per-installment one).
- Tax is recalculated per payment rather than charged once up front.
- The native "Partially paid" badge is no longer the status source; the ledger and the admin dashboard are.

The long-term ask: a platform primitive to *add a transaction of \$X to an existing order*. With it, POS keeps fully native flows, orders keep true totals, and most of this app's workaround code (cart transform, deposit discount, ledger gymnastics) disappears. This prototype is both the interim solution and the evidence for that ask.

05 What was built — one app, six surfaces

Surface	Component
Online store	Theme app embed: "Pay With Reserve" + down-payment modal
Checkout	Cart Transform function (deposit repricing) + breakdown UI block
Customer accounts	Installment plan card + "Make a payment" modal
POS	Tile + modal: new layaway, pay/cancel installments, ledger lookup
Admin	Layaway dashboard (active, outstanding, collected, per-order ledger)
Backend	Ledger engine (webhook-driven, idempotent), installment API, fulfillment holds

Key gotchas encountered: `write_order_edits` is a distinct scope from `write_orders`; order edits must use the presentment currency (CAD ≠ shop USD on our demo store); poll after `orderEditCommit` (no read-after-write guarantee); guard against stacking a second pending installment on an uncollected one.

Layaway prototype · Shopify VIP demo environment · companion diagrams: *layaway_flows.png* (deposit + installment flows, POS explainer) and *layaway_architecture.png* (component architecture) · full design docs in the app repo under /docs